

NODO CHICAMOCHA DC

Monitoreo IoT de un centro de datos urbano sobre Azure IoT Central



PARCIAL 1 · Escenario IoT Central con flota heterogénea de 10 dispositivos

Escenario 5.2 — Centro de datos · Expediente del cliente

Cliente (caso)	Chicamocha Data Center S.A.S. (empresa ficticia del caso)
Autor	Juan Rojas Guerrero
Asignatura	IoT + Cloud + Sistemas Distribuidos
Institución	Universidad Autónoma de Bucaramanga (UNAB)
Periodo	Parcial 1 · 2026-II
Sitio	Bucaramanga (7.119° N, 73.123° O, ~960 m s. n. m.)
Versión del documento	1.0 · 2026-09-28



Historial de versiones

Versión	Fecha	Autor	Cambio
0.1	2026-09-28	Juan Rojas Guerrero	Selección del escenario 5.2, catálogo de 10 orígenes y arquitectura.
0.2	2026-09-28	Juan Rojas Guerrero	Plantillas DTDL v1 (8 interfaces), Rules R01-R14 y tablas de parámetros.
0.3	2026-09-28	Juan Rojas Guerrero	Códigos de los 10 orígenes v1.0.0, herramienta de análisis de 4 días.
1.0	2026-09-28	Juan Rojas Guerrero	Expediente completo; secciones de evidencia listas para capturas.

Versiones de artefactos (Device Template y scripts)

Artefacto	Versión	Detalle
Device Templates (8 × DTDL v2)	1	templates/*.json — modelId ...;1
common/iotc_common.py · sdk_device.py	1.0.0	Utilidades DPS/SAS y envoltorio SDK
dev01 rack_a.py	1.0.0	Python SDK MQTT
dev02 sketch.ino (Rack B)	1.0.0	Wokwi ESP32 #1
dev04 aisle.py	1.0.0	Python SDK MQTT-WS
dev05 weather_bridge.py	1.0.0	OpenWeather / Open-Meteo
dev06 airquality_bridge.py	1.0.0	Open-Meteo Air Quality
dev07 leak_paho.py	1.0.0	paho-mqtt crudo
dev08 replay.js	1.0.0	Node.js AMQP + CSV
dev09 pdu_rest.py	1.0.0	HTTPS REST
dev10 sketch.ino (Puerta)	1.0.0	Wokwi ESP32 #2
tools/analisis_4dias.py	1.0.0	Estadística de los 4 días

Regla de versionado: cambiar un campo de una plantilla publicada exige nueva versión del modelo (;1 → ;2) y migrar los dispositivos en IoT Central; los scripts siguen SemVer y reportan su versión como propiedad `firmwareVersion`, visible en la vista del dispositivo.



Contenido

Historial de versiones	2
Versiones de artefactos (Device Template y scripts)	2
Contenido	3
1. Resumen ejecutivo	4
2. Escenario y criterios de diseño	4
2.1 Por qué el escenario 5.2	4
2.2 Requisitos del cliente → decisión de diseño	4
2.3 Envolvente térmica de referencia (ASHRAE A1)	4
3. Arquitectura de referencia	5
3.1 Capa de telecomunicaciones	6
4. Digital Twin: plantillas de dispositivo	6
4.1 Vistas por plantilla	7
5. Catálogo único de 10 dispositivos	7
5.1 Por qué cada origen es distinto y cómo se provisiona	8
6. Tablas de parámetros por variable	9
7. Rules y alertas	10
8. Asincronía, desconexión y operación en línea	11
8.1 Asincronía por diseño	11
8.2 Desconexión controlada y reconexión	12
8.3 Logs de los dos códigos de la sustentación	12
9. Ventana de 4 días no continuos y comparativa	13
9.1 Guía de lectura operativa (qué explicar de cada extremo)	13
10. Dashboard tipo cuarto de control	14
11. Guion de la sustentación presencial	15
11.1 Plan de contingencia	15
11.2 Preguntas probables y respuesta preparada	16
12. Limitaciones percibidas de IoT Central	16
13. Repositorio y seguridad	16
13.1 Ejecución rápida	17
14. Anexo A · Despliegue paso a paso en IoT Central	17
15. Anexo B · Trazabilidad con la rúbrica	17
16. Anexo C · Referencias	18



1. Resumen ejecutivo

Chicamocha Data Center S.A.S. (cliente del caso) opera una sala blanca urbana en Bucaramanga con una fila de tres racks. Solicita un **demo funcional** de monitoreo en Azure IoT Central que un operador no programador pueda leer desde un cuarto de control: mapa térmico de los tres armarios, alarmas de agua, humo y energía, estado de contención de pasillo frío, acceso a la sala y las condiciones exteriores que habilitan el free-cooling.

La solución despliega **10 dispositivos con 10 orígenes de envío distintos** (Python SDK, Wokwi ESP32, simulador nativo del Digital Twin, Python por WebSockets, feed meteorológico, API pública de calidad de aire, cliente MQTT explícito, replay CSV por AMQP, puente HTTP/REST y una segunda instancia Wokwi), cuatro protocolos de transporte (MQTT, MQTT-WS, AMQP y HTTPS) y **siete cadencias** distintas (15 s a 15 min, más envío por evento). Ocho plantillas DTDL modelan el Digital Twin; catorce Rules separadas por categoría generan las alertas.

- **Indispensables del escenario cubiertos:** temperatura y humedad en los racks A, B y C (DEV-01/02/03); control room con mapa térmico y alarmas de agua (DEV-07), humo (DEV-08) y energía (DEV-09).
- **Orígenes obligatorios presentes:** Digital Twin/simulador (DEV-03), Wokwi (DEV-02), Python (DEV-01), API pública (DEV-06), feed meteorológico equivalente a Atlas Weather (DEV-05).
- **Nodos en vivo de la sustentación:** DEV-01 (Python en el portátil 1) y DEV-02 (Wokwi en el portátil 2). Ambos implementan el comando `simulateOutage` para reproducir la desconexión y reconexión.
- **Evidencia real:** las secciones 8 a 10 reciben las capturas de IoT Central y las estadísticas del export de los 4 días. Si aparece un recuadro **PENDIENTE**, esa evidencia aún no se ha cargado.

2. Escenario y criterios de diseño

2.1 Por qué el escenario 5.2

El centro de datos tiene criterios de aceptación cuantitativos publicados (ASHRAE TC 9.9 para la envolvente térmica de equipos de TI clase A1 y TIA-942 como marco académico de disponibilidad). Esto permite que cada umbral de Rule se justifique con una referencia y no con una cifra arbitraria. Además, es un sitio urbano con fibra, lo que simplifica la capa de telecomunicaciones y concentra el esfuerzo en la heterogeneidad de orígenes.

2.2 Requisitos del cliente → decisión de diseño

Requisito	Decisión
Tres racks con T y HR (indispensable)	Plantilla única RackThermal para A, B y C; cada rack con un origen distinto (Python, Wokwi, simulador) para comparar comportamiento real vs. simulado.
Mapa térmico de los tres armarios	Tile de mapa de calor / KPI con <code>tempIntake</code> y <code>tempExhaust</code> de A/B/C en el dashboard.
Alarmas de agua, humo y energía	Rules críticas R06, R08 y R10 con acción de correo y webhook; bloque de alertas en el control room.
Contención de pasillos	Presión diferencial (Pa) frío-caliente; una caída bajo 2 Pa indica puerta de contención abierta o paneles ciegos faltantes.
Free-cooling	Clima exterior real de Bucaramanga (feed meteorológico) y calidad del aire de admisión (API pública) para decidir apertura de dampers.
Seguridad física	Puerta con contacto magnético y lector de credencial; Rules de puerta retenida e intentos rechazados.

2.3 Envolvente térmica de referencia (ASHRAE A1)

Rango recomendado de temperatura de entrada 18–27 °C; permisible A1 15–32 °C; punto de rocío recomendado –9 a 15 °C con humedad relativa ≤ 60 %. De aquí salen R01 (> 27 °C aviso), R02 (> 32 °C crítico), R03 (HR > 60 % o < 20 %) y R05 (punto de rocío > 15 °C).

3. Arquitectura de referencia

La figura 1 muestra la flota completa en cuatro capas. El color del borde de cada dispositivo indica su protocolo de transporte hacia Azure. Todos los enlaces viajan cifrados con TLS 1.2 y se autentican con tokens SAS derivados de la clave del grupo de conexión de IoT Central; ninguna credencial queda en el código fuente.

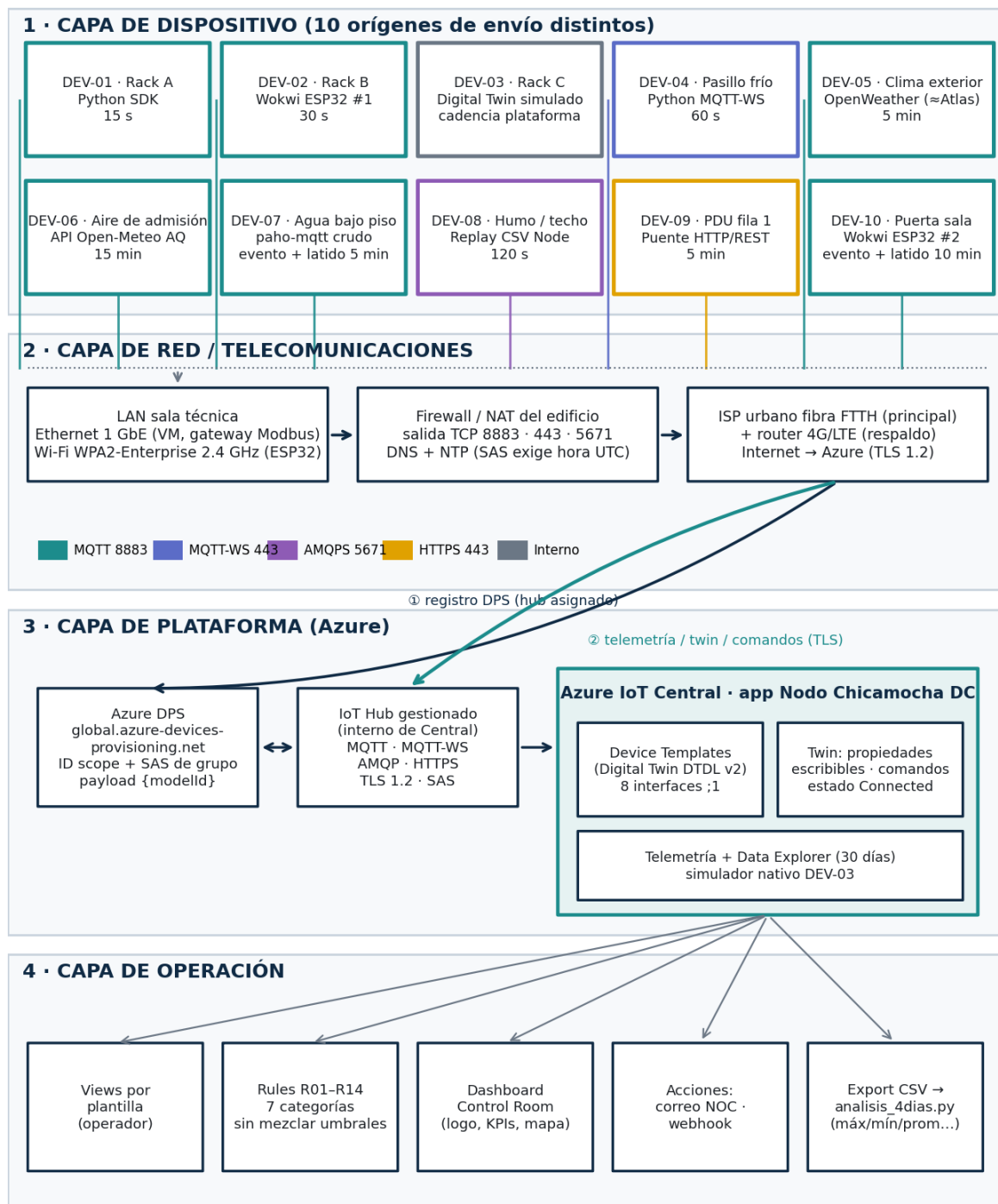


Figura 1. Arquitectura de referencia de la flota (dispositivo, red, plataforma, operación).



3.1 Capa de telecomunicaciones

Protocolo	Puerto	Dispositivos	Autenticación / sesión	Justificación
MQTT 3.1.1 / TLS 1.2	8883/TCP	01, 02, 05, 06, 07, 10	SAS en password; keep-alive 60-120 s; sesión persistente	Bidireccional (comandos, twin), bajo overhead; estándar de IoT Hub.
MQTT sobre WebSockets	443/TCP	04	SAS; túnel wss://	Redes corporativas que solo permiten 443; valida la ruta alterna.
AMQP 1.0 / TLS	5671/TCP	08	SAS (CBS); enlace multiplexado	Protocolo de gateways y backends; tercer transporte de la flota.
HTTPS REST	443/TCP	09	SAS por petición; sin sesión	Gateways simples/Modbus que publican por lotes; sin comandos en tiempo real.
Interno	—	03	Gestionado por IoT Central	Simulador nativo del Digital Twin.

Elemento de red	Especificación	Riesgo que mitiga
Acceso local	Ethernet 1 GbE en la sala para VM y gateway; Wi-Fi WPA2-Enterprise 2,4 GHz para ESP32 (en la simulación: SSID Wokwi-GUEST)	Separación de VLAN de monitoreo respecto a la red de producción
Salida a Internet	Fibra FTTH del ISP urbano + router 4G/LTE de respaldo con conmutación automática	Caída del ISP: la telemetría sigue saliendo por LTE
Firewall	Solo salida hacia *.azure-devices.net y *.azure-devices-provisioning.net en 8883, 443 y 5671; sin puertos de entrada	Superficie de ataque mínima: el dispositivo siempre inicia la conexión
DNS y NTP	Resolución de los hosts de Azure; NTP obligatorio porque el token SAS tiene expiración en UTC	Tokens rechazados por desfase de reloj
DPS	global.azure-devices-provisioning.net con ID scope de la app; payload {modelId}	Asignación automática de plantilla y del hub interno

4. Digital Twin: plantillas de dispositivo

Cada plantilla es una interfaz DTDL v2 generada desde docs/catalogo.py por templates/gen_dtdl.py e importada en IoT Central (Device templates → IoT device → Import a model). Todas incluyen las propiedades comunes firmwareVersion, sourceOrigin y lastBoot para que el operador vea qué origen y qué versión de código alimenta cada gemelo. Las telemetrías con tipo semántico (Temperature, RelativeHumidity, Pressure, Power, Energy...) llevan su unidad en el modelo.

Plantilla / modelId	Telemetría	Escribibles	Comandos	DEV
Rack térmico (A/B/C) dtmi:chicamochadc:RackThermal;1	tempIntake, tempExhaust, humidity, deltaT, overTemp	sampleIntervalS, intakeAlarmC	setFanBoost, simulateOutage	01, 02, 03
Pasillo frío / contención dtmi:chicamochadc:ColdAisle;1	tempAisle, humidity, diffPressure, dewPoint	sampleIntervalS	simulateOutage	04
Clima exterior (free-cooling) dtmi:chicamochadc:OutdoorWeather;1	tempOutdoor, humidity, dewPoint, pressure, windSpeed, rain1h, cloudCover, freeCoolingAvailable, sourceAgeMin	sampleIntervalS	simulateOutage	05
Calidad de aire de admisión dtmi:chicamochadc:AirIntakeQuality;1	pm25, pm10, aqi, sourceAgeMin	sampleIntervalS	simulateOutage	06
Detección de agua bajo piso dtmi:chicamochadc:WaterLeak;1	leakDetected, leakZone, floorHumidity, eventReason	—	resetLeakLatch, injectLeakTest	07
Detección de humo / incendio dtmi:chicamochadc:SmokeFire;1	smokeObscuration, ceilingTemp, smokeAlarm, sirenSilenced, replayRow	—	silenceAlarm	08
PDU / energía de fila dtmi:chicamochadc:PduEnergy;1	activePowerKw, currentA, voltageV, powerFactor, energyKwh, energyTotalKwh, loadPercent, breakerClosed	—	—	09
Puerta / control de acceso dtmi:chicamochadc:AccessDoor;1	doorOpen, doorOpenSeconds, accessGranted, accessDenied, eventReason	doorHeldAlarmS	unlockDoor	10



4.1 Vistas por plantilla

- **Overview (operador):** último valor de cada variable, gráfico de 24 h y estado de conexión.
- **Properties:** formulario con las propiedades escribibles (umbral local, intervalo de muestreo) y las de solo lectura (versión, origen, marca de tiempo de la fuente).
- **Commands:** generada por la plataforma a partir de los comandos del modelo (setFanBoost, simulateOutage, unlockDoor, resetLeakLatch...).
- La identidad visual (logo, colores) se configura en Customization → Appearance de la aplicación.

PENDIENTE · captura real — guarde la imagen como evidencias/01_plantillas.png y ejecute python docs/build_pdf.py.

Debe mostrar: la lista de Device templates con las 8 plantillas en estado Published.

Evidencia 1. Plantillas publicadas en IoT Central.

PENDIENTE · captura real — guarde la imagen como evidencias/17_simulado_props.png y ejecute python docs/build_pdf.py.

Debe mostrar: el dispositivo dc-rack-c-sim con telemetría periódica y la propiedad intakeAlarmC modificada desde la vista Properties.

Evidencia 2. Rack C simulado (Digital Twin) con propiedad editada.

5. Catálogo único de 10 dispositivos

Una sola tabla, diez filas y diez orígenes distinguibles. La flota no se parte en bloques: cada fila tiene un código o un feed diferente, y todos los orígenes obligatorios de la sección 3 del enunciado están presentes.

#	ID en Central	Zona	Origen de envío	Protocolo	Intervalo	Variables	Datasheet citado
01	dc-rack-a-py	Rack A (fila 1)	Python #1 · azure-iot-device	DPS + MQTT/TLS 8883	15 s	tempIntake, tempExhaust, humidity, deltaT	Sensirion SHT45; Analog Devices (Maxim) DS18B20
02	dc-rack-b-wokwi	Rack B (fila 1)	Wokwi ESP32 #1 (Arduino)	MQTT/TLS 8883 (PubSubClient + SAS en el borde)	30 s	tempIntake, tempExhaust, humidity, deltaT	Aosong DHT22/AM2302; DS18B20
03	dc-rack-c-sim	Rack C (fila 1)	Digital Twin · simulador nativo	Interno de IoT Central	Fija de la plataforma	tempIntake, tempExhaust, humidity, deltaT	Mismo modelo que Rack A/B (referencia SHT45 / DS18B20)
04	dc-aisle-cold-pyws	Pasillo frío / contención	Python #2 · protocolo distinto	DPS + MQTT sobre WebSockets TLS 443	60 s	tempAisle, humidity, diffPressure, dewPoint	Sensirion SHT45; Sensirion SDP810-125Pa
05	dc-weather-owm	Cubierta / clima exterior	Feed meteorológico (equivalente Atlas Weather)	HTTPS→API OpenWeather · MQTT 8883→Central	5 min	tempOutdoor, humidity, dewPoint, pressure, windSpeed, rain1h, cloudCover	Referencia de estación: Davis Vantage Pro2 (ISS 6152)
06	dc-airq-api	Toma de aire del free-cooling	API pública (calidad de aire)	HTTPS→Open-Meteo AQ · MQTT 8883→Central	15 min	pm25, pm10, aqi, sourceAgeMin	Referencia: Plantower PMS5003
07	dc-leak-paho	Piso técnico (cable perimetral)	Cliente MQTT explícito (paho-mqtt)	DPS REST + MQTT/TLS 8883 crudo	Evento + latido 5 min	leakDetected, leakZone, floorHumidity	Cable de fuga por conductividad (tipo RLE SeaHawk/LD310); Sensirion SHT31
08	dc-smoke-replay	Techo sala / cuarto UPS	Replay de CSV histórico (Node.js)	DPS + AMQPS 5671	120 s	smokeObscuration, ceilingTemp, smokeAlarm	Detector fotoeléctrico listado UL 268; DS18B20 (techo)



#	ID en Central	Zona	Origen de envío	Protocolo	Intervalo	Variables	Datasheet citado
09	dc-pdu-rest	PDU de fila 1	Puente HTTP/REST (gateway Modbus)	DPS REST + HTTPS 443 POST	5 min	activePowerKw, currentA, voltageV, powerFactor, energyKwh	Eastron SDM630-Modbus V2
10	dc-door-wokwi	Puerta de sala blanca	Wokwi ESP32 #2 (segunda instancia)	MQTT/TLS 8883	Evento + latido 10 min	doorOpen, doorOpenSeconds, accessGranted, accessDenied	Contacto magnético MC-38; lector RFID 13,56 MHz (MFRCS22)

5.1 Por qué cada origen es distinto y cómo se provisiona

#	Qué lo hace distinto	Provisionamiento	Dónde corre	Código
01	SDK oficial de Python, sesión MQTT persistente, comandos y propiedades escribibles.	DPS con SDK Python, clave simétrica derivada	Portátil 1 (EN VIVO)	devices/dev01_rack_a_python_mqtt/rack_a.py
02	Firmware C++ en microcontrolador simulado; firma SAS con mbedTLS; actuador (relé) por comando.	DPS REST (tools/provision.py) → hub + clave en secrets.h	Portátil 2 · navegador (EN VIVO)	devices/dev02_rack_b_wokwi/sketch.ino
03	Dispositivo simulado creado sobre la plantilla; no hay código del grupo.	Creado en Central con Simulate = Yes	IoT Central (nube)	devices/dev03_rack_c_simulador/README.md
04	Transporte WebSocket 443 (apto para redes que bloquean 8883); cálculo de punto de rocío en el borde.	DPS con SDK Python, clave simétrica derivada	VM / portátil (segundo plano)	devices/dev04_pasillo_python_mqttws/aisle.py
05	Datos meteorológicos reales de Bucaramanga; doble marca de tiempo (fuente / ingestión); respaldo Open-Meteo.	DPS con SDK Python, clave simétrica derivada	VM (segundo plano)	devices/dev05_clima_openweather/weather_bridge.py
06	Otro proveedor y otro dominio de datos (calidad de aire, modelo CAMS), fuente horaria.	DPS con SDK Python, clave simétrica derivada	VM (segundo plano)	devices/dev06_aire_openmeteo/airquality_bridge.py
07	Sin SDK: tópicos, SAS y respuestas de comando implementados a mano; envío dirigido por evento.	DPS REST desde el script (payload modelId)	VM (segundo plano)	devices/dev07_agua_paho_mqtt/leak_paho.py
08	Otro lenguaje (JavaScript) y otro protocolo (AMQP); reproduce un perfil de prueba de 24 h.	DPS por AMQP con SDK de Node.js	VM (segundo plano)	devices/dev08_humo_node_amqp_replay/replay.js
09	Sin sesión persistente: cada muestra es un POST HTTPS firmado; sin comandos (limitación HTTP).	DPS REST desde el script (payload modelId)	VM (segundo plano)	devices/dev09_pdu_http_rest/pdu_rest.py
10	Circuito y sketch distintos a DEV-02; entradas digitales dirigidas por evento; comando de cerradura.	DPS REST (tools/provision.py) → hub + clave en secrets.h	Navegador (contingencia en vivo)	devices/dev10_puerta_wokwi/sketch.ino

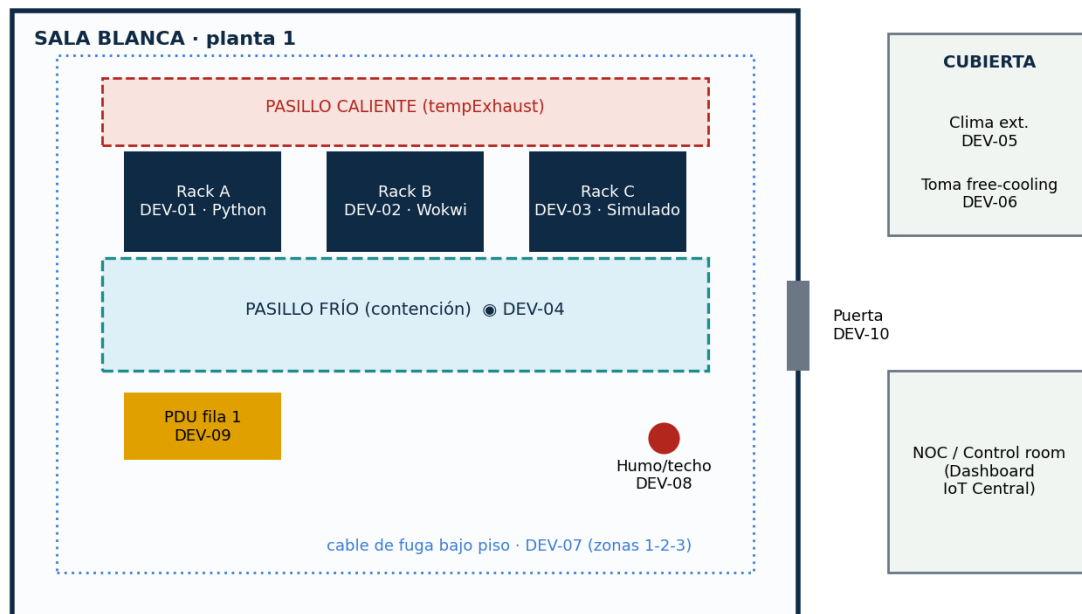


Figura 2. Mapa de zonas: relación dispositivo ↔ lugar físico (también se usa como tile de imagen del dashboard).

6. Tablas de parámetros por variable

Cada variable queda anclada a un sensor o equipo real. La columna *Código* muestra los valores exactos usados en los scripts (base, amplitud del perfil diario, límites de recorte y offset de calibración) o, en los orígenes de API, la ventana de validación que descarta datos físicamente imposibles. Los rangos del fabricante deben contrastarse con la hoja de datos vigente de cada referencia (anexo C) antes de la entrega final.

DEV	Variable	Unidad	Rango datasheet	Rango operativo	Precisión	Umbral de Rule	Código (min/max/offset)
01-03	tempIntake	°C	−40...125 (SHT45) / −40...80 (DHT22)	18...27 (ASHRAE A1 recom.)	±0,1 °C / ±0,5 °C	> 27 aviso · > 32 crítico	base 22,5 ± 1,8; clamp 18...30; offset 0,0
01-03	tempExhaust	°C	−55...125 (DS18B20)	28...45	±0,5 °C (−10...85 °C)	> 45 aviso	intake + ΔT 10...14; clamp 26...46; offset −0,3
01-03	humidity	%HR	0...100 (SHT45 / DHT22)	20...60	±1,0 %HR / ±2-5 %HR	> 60 aviso · < 20 aviso (ESD)	base 45 ± 5; clamp 30...58; offset 0,0
04	tempAisle	°C	−40...125 (SHT45)	18...27	±0,1 °C	> 27 aviso	base 20,5 ± 1,2; clamp 17...27; offset 0,0
04	diffPressure	Pa	−125...+125 (SDP810-125Pa)	+2...+10	±3 % de lectura (típ.)	< 2 aviso (fuga de contención)	base 4,5 σ0,35; eventos 0,3...1,6; clamp 0...10; offset cero 0,2
04	dewPoint	°C	calculado (Magnus)	−9...15 (ASHRAE)	≈ ±0,3 °C (propagado)	> 15 aviso	Magnus a=17,62 b=243,12
05	tempOutdoor	°C	−40...65 (Davis VP2)	17...32 (Bucaramanga)	±0,3 °C	> 30 info (sin free-cooling)	validación −10...50
05	humidity (ext.)	%HR	1...100 (Davis VP2)	50...95	±2 %HR (0-90 %)	—	validación 0...100
05	windSpeed	m/s	0,5...89 (Davis VP2)	0...8	±1 m/s o ±5 %	> 15 aviso (cubierta)	validación 0...60



DEV	Variable	Unidad	Rango datasheet	Rango operativo	Precisión	Umbral de Rule	Código (min/max/offset)
05	rain1h	mm	0,2 mm/resolución (Davis VP2)	0...40	±4 % (típ.)	> 20 aviso (filtraciones)	validación 0...150
06	pm25	µg/m ³	0...500 efectivo (PMS5003)	5...35	±10 µg/m ³ (0-100)	> 35,4 aviso (cerrar dampers)	validación 0...500
06	pm10	µg/m ³	0...500 efectivo (PMS5003)	10...60	±10 µg/m ³ (0-100)	> 154 aviso	validación 0...600
06	aqi	—	0...500 (US EPA)	0...100	derivado	> 100 aviso	validación 0...500
07	leakDetected	bool	contacto seco (cable conductivo)	false	binaria	= true crítico	prob. 1,5e-4 por sondeo de 5 s
07	floorHumidity	%HR	0...100 (SHT31)	30...70	±2 %HR	> 80 aviso	base 48; clamp 30...95; offset 0,0
08	smokeObscuration	%/ft	0,5...4,0 sensibilidad (UL 268)	0...0,2	según listado	> 1,5 pre-alarma	CSV 720 filas; máx. 3,6
08	ceilingTemp	°C	-55...125 (DS18B20)	20...35	±0,5 °C	> 57 crítico (detector térmico)	CSV 23,4...34,6
08	smokeAlarm	bool	salida de alarma del detector	false	binaria	= true crítico	obscuration ≥ 2,5 %/ft
09	currentA	A	0,5...100 (SDM630 directo)	5...25,6	±0,5 % (clase 1)	> 25,6 aviso (80 % de 32 A)	base 16 ± 4; clamp 2...32; offset 0,0
09	voltageV	V	100...289 F-N (SDM630)	114...140 (127 V ±10 %)	±0,5 %	< 114 o > 140 aviso	127 σ1,5; clamp 114...140
09	powerFactor	—	-1...1 (SDM630)	0,90...1,00	±1 %	< 0,90 aviso	0,95 σ0,015; clamp 0,80...1,00
09	activePowerKW	kW	calculado 3·V·I·FP	3...9	clase 1	> 9,5 aviso	derivado
09	energyKwh	kWh	registro de energía (SDM630)	0,2...0,8 por 5 min	clase 1	— (sumatoria diaria)	kW × 300 s / 3600
10	doorOpen	bool	gap ≤ 15-25 mm (MC-38)	false	binaria	—	slide switch Wokwi
10	doorOpenSeconds	s	contador de firmware	0...60	1 s	> 120 aviso (puerta retenida)	reporte c/30 s si abierta
10	accessDenied	eventos	lector 13,56 MHz (MFRC522)	0...2 por hora	conteo	suma > 3 en 5 min aviso	pulsador rojo

Nota sobre DEV-01/02/03: se usa la misma plantilla y los mismos umbrales para los tres racks. La precisión del DHT22 (Rack B, Wokwi) es menor que la del SHT45 (Rack A); por eso el umbral de aviso se evalúa sobre el promedio de 5 minutos y no sobre muestras individuales.

7. Rules y alertas

Las Rules están separadas por categoría y ningún umbral se comparte entre categorías. Las de variables analógicas usan agregación temporal de 5 min para evitar alertas por ruido; las binarias (fuga, humo, breaker) se evalúan sin agregación porque su latencia importa más que su estabilidad.

ID	Categoría	Nombre	Plantilla	Condición	Severidad	Acción
R01	Térmica	Rack · entrada alta	RackThermal	tempIntake > 27 (promedio 5 min)	Aviso	Correo NOC + webhook
R02	Térmica	Rack · entrada crítica	RackThermal	tempIntake > 32 (sin agregación)	Crítico	Correo NOC + Power Automate → setFanBoost
R03	Térmica	Rack · humedad fuera de rango	RackThermal	humidity > 60 o < 20 (promedio 5 min)	Aviso	Correo
R04	Contención	Pérdida de presión diferencial	ColdAisle	diffPressure < 2 (promedio 5 min)	Aviso	Correo



ID	Categoría	Nombre	Plantilla	Condición	Severidad	Acción
R05	Contención	Riesgo de condensación	ColdAisle	dewPoint > 15	Aviso	Correo
R06	Agua	Fuga bajo piso	WaterLeak	leakDetected = true	Crítico	Correo + webhook
R07	Humo	Pre-alarma de humo	SmokeFire	smokeObscuration > 1,5	Aviso	Correo
R08	Humo	Alarma de incendio	SmokeFire	smokeAlarm = true o ceilingTemp > 57	Crítico	Correo + webhook
R09	Energía	Sobrecarga de PDU	PduEnergy	currentA > 25,6 (promedio 5 min)	Aviso	Correo
R10	Energía	Breaker abierto / FP bajo	PduEnergy	breakerClosed = false · powerFactor < 0,9	Crítico / Aviso	Correo + webhook
R11	Acceso	Puerta retenida	AccessDoor	doorOpenSeconds > 120	Aviso	Correo
R12	Acceso	Intentos rechazados	AccessDoor	suma(accessDenied) > 3 en 5 min	Aviso	Correo
R13	Exterior	Aire exterior contaminado	AirIntakeQuality	pm25 > 35,4 o aqi > 100	Aviso	Correo (cerrar dampers)
R14	Exterior	Sin free-cooling por calor	OutdoorWeather	tempOutdoor > 30 (promedio 15 min)	Info	Correo

PENDIENTE · captura real — guarde la imagen como evidencias/04_rules.png y ejecute python docs/build_pdf.py.
Debe mostrar: la lista de Rules R01-R14 habilitadas.

Evidencia 3. Rules configuradas y habilitadas.

8. Asincronía, desconexión y operación en línea

8.1 Asincronía por diseño

La flota usa siete cadencias: 15 s, 30 s, 60 s, 120 s, 5 min, 15 min y la del simulador, más dos dispositivos que envían solo ante eventos con latido periódico. Esto supera el mínimo de tres intervalos. La figura 3 es el diseño que queda en el código; la evidencia 4 debe mostrar ese mismo patrón en los datos de IoT Central.

Diseño de asincronía de la flota (cadencias configuradas en código)

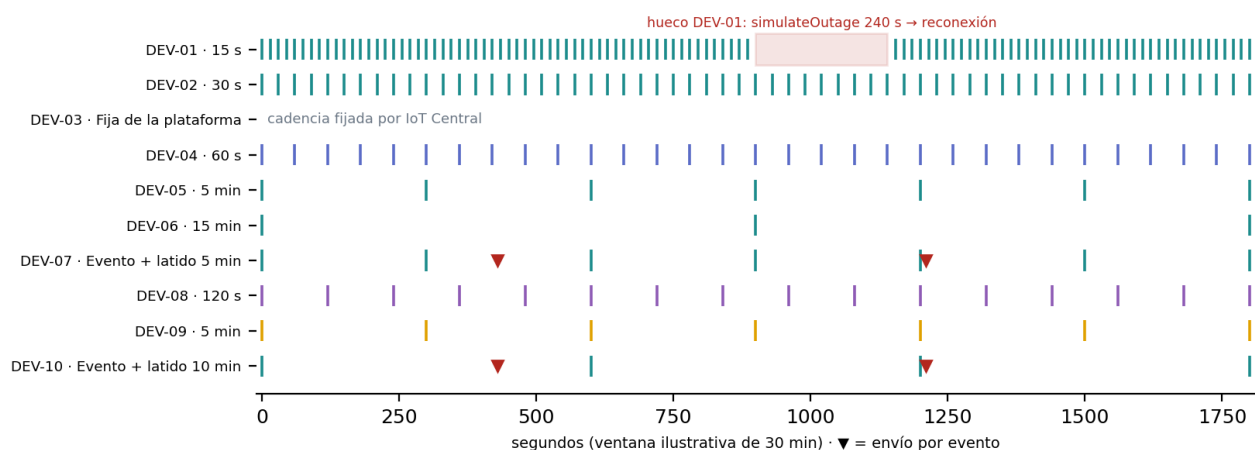


Figura 3. Diseño de cadencias (ilustrativo, derivado de los intervalos del código; no es una captura).

PENDIENTE · captura real — guarde la imagen como evidencias/10_asincronia.png y ejecute python docs/build_pdf.py.
Debe mostrar: un gráfico de Data Explorer con al menos tres dispositivos de intervalos distintos (por ejemplo DEV-01 a 15 s, DEV-04 a 60 s y DEV-09 a 5 min).

Evidencia 4. Asincronía observada en IoT Central.

8.2 Desconexión controlada y reconexión

Procedimiento reproducible (se ejecuta en la sustentación con DEV-01 o DEV-02):

- En IoT Central → dispositivo → Commands → simulateOutage con valor 240.
- El dispositivo responde 200, cierra su sesión MQTT y deja de publicar; el log muestra === DESCONEXIÓN CONTROLADA 240 s === y Central lo marca **Disconnected**.
- A los 240 s el código vuelve a abrir la sesión (=== RECONEXIÓN ===), el estado pasa a **Connected** y la serie retoma. En el gráfico queda un hueco de ≈ 4 min.
- Alternativas equivalentes: Ctrl+C del script (apagado controlado), detener la simulación en Wokwi o desconectar el Wi-Fi del portátil (fallo de red). El SDK y paho reconectan con backoff exponencial.

PENDIENTE · captura real — guarde la imagen como evidencias/11_desconexion_hueco.png y ejecute python docs/build_pdf.py.

Debe mostrar: la serie de DEV-01 o DEV-02 con el hueco del simulateOutage y la telemetría retomada.

Evidencia 5. Hueco en la serie y reconexión.

PENDIENTE · captura real — guarde la imagen como evidencias/12_estado_disconnected.png y ejecute python docs/build_pdf.py.

Debe mostrar: la lista Devices con un dispositivo en Disconnected y los demás en Connected.

Evidencia 6. Estado Disconnected en la lista de dispositivos.

8.3 Logs de los dos códigos de la sustentación

Ambos códigos imprimen el ciclo completo Connecting → Connected → TX #n → comando → Disconnected → reconexión. El script Python además lo guarda en logs/dc-rack-a-py.log. Extracto esperado del formato:

```
2026-10-01 09:14:02 | INFO | dc-rack-a-py | Connecting | DPS dc-rack-a-py | transporte=MQTT:8883
2026-10-01 09:14:04 | INFO | dc-rack-a-py | Connected | hub=iotc-...azure-devices.net
2026-10-01 09:14:04 | INFO | dc-rack-a-py | TX #1 {'tempIntake': 22.4, 'tempExhaust': 34.9, ...}
(formato de ejemplo – la evidencia real es la captura 13)
```

PENDIENTE · captura real — guarde la imagen como evidencias/13_log_python_rack_a.png y ejecute python docs/build_pdf.py.

Debe mostrar: la consola con Connecting, Connected, varias líneas TX y la respuesta a un comando.

Evidencia 7. Log de sesión de rack_a.py (portátil 1).

PENDIENTE · captura real — guarde la imagen como evidencias/14_log_wokwi_rack_b.png y ejecute python docs/build_pdf.py.

Debe mostrar: [MQTT] Connected, líneas [TX #n] OK y la recepción de setFanBoost.

Evidencia 8. Monitor serie de Wokwi, Rack B (portátil 2).

PENDIENTE · captura real — guarde la imagen como evidencias/15_comando_wokwi.png y ejecute python docs/build_pdf.py.

Debe mostrar: la ventana de historial de comandos con setFanBoost respondido por dc-rack-b-wokwi.

Evidencia 9. Respuesta del comando en IoT Central.

PENDIENTE · captura real — guarde la imagen como evidencias/16_api_doble_timestamp.png y ejecute python docs/build_pdf.py.

Debe mostrar: DEV-05 o DEV-06 con la propiedad lastSourceTimestamp, la telemetría sourceAgeMin y la hora de ingestión del mensaje.

Evidencia 10. Orígenes de API con marca de tiempo de fuente y de ingestión.

9. Ventana de 4 días no continuos y comparativa

Días analizados: **pendiente**. Elija cuatro fechas no consecutivas con la flota encendida y exporte los datos de Data Explorer antes de 30 días (retención de IoT Central). Sugerencia de calendario a partir de la fecha de este documento: un día de semana normal, un día con mantenimiento programado (dispara pre-alarma de humo o apertura de puerta), un fin de semana y un día con la prueba de desconexión controlada.

Método: para cada día, dispositivo y variable se calcula máximo, mínimo, promedio y recuento de muestras; la sumatoria se reporta solo donde tiene significado físico (energía del intervalo → kWh/día, conteos de acceso, muestras en alarma). El recuento también valida la asincronía: a 15 s DEV-01 debería aportar $\approx 5\,760$ muestras diarias; a 5 min DEV-09 ≈ 288 ; una cifra menor delata huecos.

Variable	Dispositivo	Día 1	Día 2	Día 3	Día 4	Indicador
templntake	Rack A/B/C	—	—	—	—	máx / mín / prom / n
humidity	Rack A/B/C	—	—	—	—	máx / mín / prom / n
diffPressure	Pasillo frío	—	—	—	—	mín (contención)
currentA	PDU	—	—	—	—	máx / prom
energyKwh	PDU	—	—	—	—	Σ = kWh/día
tempOutdoor	Clima ext.	—	—	—	—	máx / mín / prom
pm25	Aire admisión	—	—	—	—	máx / prom
smokeObscuration	Humo	—	—	—	—	máx (pre-alarma)
accessDenied	Puerta	—	—	—	—	Σ = intentos/día

9.1 Guía de lectura operativa (qué explicar de cada extremo)

- **templntake máx.** ¿Coincide con la hora de mayor carga de TI o con temperatura exterior alta? Si superó 27 °C, R01 debe figurar en el bloque de alertas de ese día.
- **diffPressure mín.** Valores bajo 2 Pa corresponden a puerta de contención abierta (mantenimiento, ingreso de personal); correlacionar con doorOpen de DEV-10.
- **Σ energyKwh.** Energía de TI de la fila por día; su diferencia entre días hábiles y fin de semana mide la variación de carga.
- **tempOutdoor mín.** Solo las madrugadas de Bucaramanga bajan de 18 °C: cuantifica las horas reales de free-cooling.
- **pm25 máx.** Si supera 35,4 $\mu\text{g}/\text{m}^3$, el economizador debió cerrarse (R13); típicamente en horas pico de tráfico.
- **Recuento.** Un recuento menor al esperado por la cadencia es la medida numérica de la desconexión.

PENDIENTE · captura real — guarde la imagen como evidencias/06_data_explorer_dia1.png y ejecute `python docs/build_pdf.py`.

Debe mostrar: el gráfico de IoT Central del día 1 con las variables indispensables (templntake y humidity de A/B/C) y la fecha visible.

Evidencia 11. Data Explorer — día 1.

PENDIENTE · captura real — guarde la imagen como evidencias/07_data_explorer_dia2.png y ejecute `python docs/build_pdf.py`.

Debe mostrar: el gráfico de IoT Central del día 2 con las variables indispensables (templntake y humidity de A/B/C) y la fecha visible.

Evidencia 12. Data Explorer — día 2.



PENDIENTE · captura real — guarde la imagen como evidencias/08_data_explorer_dia3.png y ejecute `python docs/build_pdf.py`.

Debe mostrar: el gráfico de IoT Central del día 3 con las variables indispensables (templntake y humidity de A/B/C) y la fecha visible.

Evidencia 13. Data Explorer — día 3.

PENDIENTE · captura real — guarde la imagen como evidencias/09_data_explorer_dia4.png y ejecute `python docs/build_pdf.py`.

Debe mostrar: el gráfico de IoT Central del día 4 con las variables indispensables (templntake y humidity de A/B/C) y la fecha visible.

Evidencia 14. Data Explorer — día 4.

PENDIENTE · captura real — guarde la imagen como evidencias/05_alertas_4dias.png y ejecute `python docs/build_pdf.py`.

Debe mostrar: el historial de alertas con fecha, Rule y dispositivo.

Evidencia 15. Rules disparadas en la ventana de 4 días.

10. Dashboard tipo cuarto de control

Dashboard de aplicación con identidad propia (logo y nombre del escenario, no el nombre genérico de la app). La figura 4 es la maqueta de diseño que guía la construcción de los tiles; la evidencia 16 es la captura real.

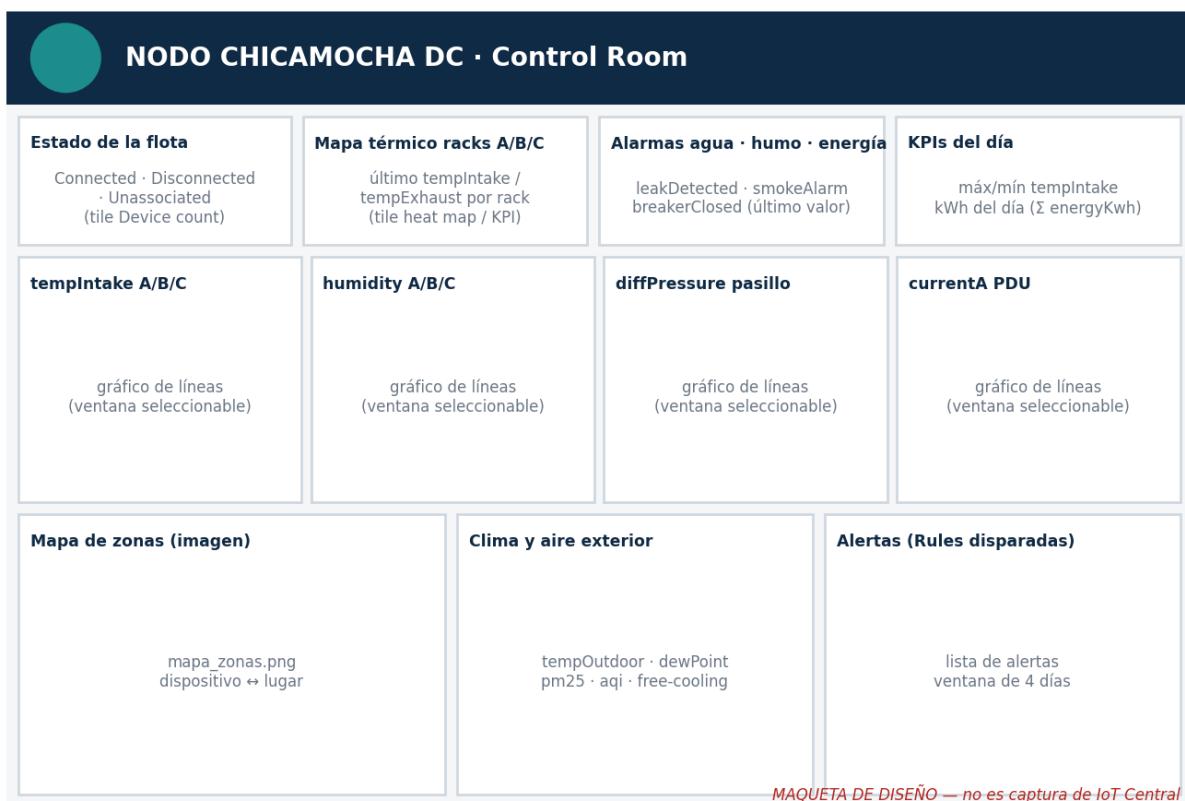


Figura 4. Maqueta de diseño del control room (no es captura de IoT Central).

Requisito del enunciado (sección 7)	Tile en IoT Central	Fuente
Logo y nombre del escenario	Appearance (logo de la app) + tile Imagen con logo y Label «Nodo Chicamocha DC · Control Room»	docs/assets/logo.png
Estado de la flota Connected/Disconnected/Unassociated	Tile de estado sobre la lista de dispositivos (o grupos de dispositivos filtrados por estado de conexión, según lo que ofrezca la versión de la app)	Estado de conexión de los 10



Requisito del enunciado (sección 7)	Tile en IoT Central	Fuente
≥ 4 gráficos de variables indispensables	Line chart: tempIntake A/B/C · humidity A/B/C · diffPressure · currentA	DEV-01...04, 09
KPIs numéricos	KPI / Last known value: máx y mín del día de tempIntake, Σ energyKwh, último pm25	DEV-01...03, 06, 09
Bloque de alertas	Tile de alertas / historial de eventos con las Rules disparadas	R01-R14
Mapa o esquema de zonas	Tile Imagen con mapa_zonas.png	Figura 2
Mapa térmico de racks (escenario 5.2)	Heat map / KPI de tempIntake y tempExhaust de A, B y C	DEV-01...03
Alarmas de agua, humo y energía (5.2)	Last known value con formato condicional: leakDetected, smokeAlarm, breakerClosed	DEV-07, 08, 09

PENDIENTE · captura real — guarde la imagen como evidencias/03_dashboard_control_room.png y ejecute `python docs/build_pdf.py`.

Debe mostrar: el dashboard completo en una sola pantalla: logo, estado de flota, cuatro gráficos, KPIs, alertas y mapa de zonas.

Evidencia 16. Dashboard del control room en IoT Central.

PENDIENTE · captura real — guarde la imagen como evidencias/02_dispositivos.png y ejecute `python docs/build_pdf.py`.

Debe mostrar: Devices con los 10 IDs del catálogo, su plantilla y el estado de conexión.

Evidencia 17. Lista de los 10 dispositivos con su plantilla y estado.

11. Guion de la sustentación presencial

Min.	Actividad	Equipo
0-2	Escenario, cliente y arquitectura (figura 1). Por qué cada origen es distinto (tabla del catálogo).	Proyector
2-4	Digital Twin: plantilla RackThermal, propiedades escribibles, comandos; Rack C simulado como gemelo de referencia.	IoT Central
4-6	Portátil 1: <code>python devices/dev01_rack_a_python_mqtt/rack_a.py</code> → Connecting / Connected / TX en vivo.	Portátil 1
6-8	Portátil 2: Wokwi Rack B ► → Connected; mover el slider del DHT22 sobre 27 °C → LED OVERTEMP y valor en Central; comando <code>setFanBoost</code> → relé.	Portátil 2
8-10	Desconexión controlada: <code>simulateOutage 120 s</code> sobre DEV-01 → Disconnected, hueco, reconexión.	IoT Central
10-12	Control room y navegación de un gráfico de los 4 días no continuos; comparativa y lectura de extremos.	IoT Central
12-15	Preguntas: umbrales, datasheets, limitaciones de IoT Central.	—

11.1 Plan de contingencia

- Si falla la red del salón: compartir datos del celular (el script usa 8883; si está bloqueado, ejecutar DEV-04, que va por 443).
- Si Wokwi no conecta (cola del simulador gratuito): usar DEV-10 (segunda instancia Wokwi) o DEV-07 (paho) en el portátil 2.
- Si las API (DEV-05/06) no responden por cuota: se muestra la evidencia histórica; el enunciado admite sustituir en vivo por el par Python + Wokwi.
- Llevar .env y secrets.h en una memoria USB (nunca en el repositorio) y el hub ya resuelto en la caché de DPS.



11.2 Preguntas probables y respuesta preparada

Pregunta	Respuesta breve
¿Por qué 27 °C y no otro valor?	Límite superior del rango recomendado ASHRAE A1 para aire de entrada; 32 °C es el máximo permisible A1 (crítico).
¿Por qué promedio de 5 min en R01?	El DHT22 tiene $\pm 0,5$ °C y ruido; se evita la alerta por una muestra aislada sin perder reacción ante una tendencia real.
¿Por qué el PDU no aparece Connected?	HTTP no mantiene sesión; IoT Central recibe la telemetría pero no un evento de conexión persistente. Es una limitación del protocolo, documentada.
¿Cómo se protege la clave?	Solo variables de entorno / secrets.h fuera de Git; la clave del dispositivo se deriva por HMAC de la del grupo y el token SAS expira en 1 h.
¿Qué pasa si el dispositivo se desconecta?	El SDK y paho reintentan con backoff; las muestras del hueco no se reenvían (se documenta como pérdida, visible en el recuento).
¿Diferencia entre Rack A, B y C?	Mismo gemelo; A es Python con perfil físico, B es firmware en ESP32 con sensores simulados y actuador, C es el simulador aleatorio de la plataforma.

12. Limitaciones percibidas de IoT Central

- **Retención:** los datos consultables en Data Explorer tienen retención limitada (30 días); para la ventana de 4 días hay que exportar antes o configurar Data Export continuo a Blob Storage.
- **Simulador nativo:** genera valores aleatorios dentro del esquema y con cadencia fija de la plataforma; no reproduce perfiles físicos ni correlaciones (por eso Rack C se usa solo como referencia).
- **Rules:** condiciones sobre una sola plantilla y ventanas de agregación predefinidas; no admite lógica entre dispositivos (p. ej. «dP bajo Y puerta abierta») sin un servicio externo.
- **Acciones:** una Rule no ejecuta un comando directamente; se requiere Power Automate/Logic Apps o un webhook propio.
- **HTTP:** sin comandos en tiempo real ni estado Connected persistente para dispositivos que publican por REST.
- **Dashboards:** tipos de tile fijos, sin estadística personalizada por día; por eso la comparativa de 4 días se calcula fuera (análisis_4días.py).
- **Costo y cuotas:** la facturación por dispositivo y por mensajes limita la cadencia en producción; 15 s es razonable para la demo, no necesariamente para 300 racks.

13. Repositorio y seguridad

```
nodo-chicamocha-dc/  
├─ README.md · requirements.txt · .env.example · .gitignore  
├─ common/ iotc_common.py (DPS REST, SAS, derivación de clave) · sdk_device.py (SDK + simulateOutage)  
├─ templates/ 8 × DTDL v2 (.json) · gen_dtdl.py  
├─ devices/  
│   ├── dev01_rack_a_python_mqtt/ rack_a.py  
│   ├── dev02_rack_b_wokwi/ sketch.ino · diagram.json · libraries.txt · secrets.h.example  
│   ├── dev03_rack_c_simulado/ README.md (pasos en Central)  
│   ├── dev04_pasillo_python_mqttws/ aisle.py  
│   ├── dev05_clima_openweather/ weather_bridge.py  
│   ├── dev06_aire_openmeteo/ airquality_bridge.py  
│   ├── dev07_agua_paho_mqtt/ leak_paho.py  
│   ├── dev08_humo_node_amqp_replay/ replay.js · package.json · data/humo_historico.csv  
│   ├── dev09_pdu_http_rest/ pdu_rest.py  
│   └── dev10_puerta_wokwi/ sketch.ino · diagram.json · secrets.h.example  
├─ tools/ provision.py · analisis_4días.py  
├─ docs/ catalogo.py (fuente única) · diagramas.py · build_pdf.py · assets/  
└─ evidencias/ (capturas reales) · datos_4días/ (exports de Central)
```

- **Sin secretos en claro:** ID scope y clave de grupo solo en .env; en Wokwi en secrets.h; ambos en .gitignore. Los ejemplos del repo usan marcadores.
- **Claves por dispositivo:** derivadas por HMAC-SHA256(clave de grupo, deviceId); comprometer un dispositivo no expone al grupo.



- **TLS:** validación del certificado del servidor en Python y Node (almacén del sistema). En Wokwi se puede cargar DigiCert Global Root G2 en secrets.h; si se deja vacío, el sketch usa setInsecure() y lo anuncia en el monitor serie (solo aceptable en el simulador).
- **Perfil CSV de DEV-08:** es un perfil de prueba de 24 h construido sobre los rangos de UL 268, declarado como tal; no se presenta como registro histórico de un incidente real.

13.1 Ejecución rápida

```
pip install -r requirements.txt # Python ≥ 3.10
cp .env.example .env # completar ID scope y clave de grupo
python devices/dev01_rack_a_python_mqtt/rack_a.py
python devices/dev04_pasillo_python_mqttws/aisle.py
python devices/dev05_clima_openweather/weather_bridge.py
python devices/dev06_aire_openmeteo/airquality_bridge.py
python devices/dev07_agua_paho_mqtt/leak_paho.py
python devices/dev09_pdu_http_rest/pdu_rest.py
cd devices/dev08_humo_node_amqp_replay && npm install && npm start
python tools/provision.py dc-rack-b-wokwi "dtmi:chicamochadc:RackThermal;1" # → secrets.h de Wokwi
python tools/analysis_4dias.py datos_4dias/*.csv --dias D1 D2 D3 D4
python docs/build_pdf.py
```

14. Anexo A · Despliegue paso a paso en IoT Central

#	Paso	Detalle
1	Crear la aplicación	portal de IoT Central → New application → plantilla Custom; nombre «Nodo Chicamocha DC»; plan Standard 0 o el de la suscripción de estudiante.
2	Identidad visual	Customization → Appearance: cargar docs/assets/logo.png como logo y como icono; colores #0F2A44 / #1C8C8C.
3	Plantillas	Device templates → + New → IoT device → Import a model → cada templates/*.json. Crear la vista Overview y la de Properties; Publish.
4	Grupo de conexión	Permissions → Device connection groups → SAS-IoT-Devices: copiar ID scope y clave primaria al .env (nunca al repo).
5	Dispositivo simulado	Devices → + New → plantilla Rack térmico, ID dc-rack-c-sim, Simulate = Yes.
6	Dispositivos reales	No es necesario crearlos: al ejecutar cada código, el DPS con payload modelId los registra y asocia a su plantilla. Verificar que ninguno quede Unassociated.
7	Wokwi	Ejecutar tools/provision.py para DEV-02 y DEV-10; pegar el resultado en secrets.h de cada proyecto Wokwi (pestaña nueva de archivo).
8	Rules	Rules → + New: crear R01-R14 con la condición, agregación y acción de la tabla de la sección 7.
9	Dashboard	Dashboards → + New «Control Room»: tiles según la tabla de la sección 10.
10	Ventana de 4 días	Dejar la flota en segundo plano (VM o equipo encendido) y registrar las fechas; exportar desde Data Explorer cada día elegido a datos_4dias/.
11	Documento	Guardar capturas en evidencias/ con los nombres de evidencias/README.md; ejecutar analisis_4dias.py y build_pdf.py.

15. Anexo B · Trazabilidad con la rúbrica

Indicador (peso)	Dónde se evidencia
IoT Template, setup + test (15 %)	Sección 4 (plantillas, propiedades, comandos), sección 7 (Rules), evidencias 1-3; identidad visual en Anexo A paso 2.
Datos, Digital Twin y arquitectura (20 %)	Secciones 3, 5 y 6: diagrama con telecomunicaciones, catálogo de 10, datasheets y rangos.
Heterogeneidad de orígenes (20 %)	Sección 5.1 (por qué es distinto) y sección 8 (asincronía, desconexión, operación en línea).
Ventana de 4 días y comparativa (15 %)	Sección 9: tabla máx/mín/prom/n/Σ, lectura operativa y capturas de los 4 días.
Control room y documento (15 %)	Sección 10, tablas de parámetros (sección 6), historial de versiones, sección 13 (repo limpio).
Sustentación y dos códigos en vivo (15 %)	Sección 11: guion, contingencia y respuestas preparadas; DEV-01 + DEV-02.



16. Anexo C · Referencias

- Microsoft Learn — Documentación de Azure IoT Central:
<https://learn.microsoft.com/es-es/azure/iot-central/>
- Microsoft Learn — Comunicación con IoT Hub mediante MQTT, AMQP y HTTPS; Device Provisioning Service (registro con clave simétrica y payload modelId).
- Azure IoT SDK para Python (azure-iot-device 2.x) y para Node.js (azure-iot-device, azure-iot-device-amqp, azure-iot-provisioning-device).
- Digital Twins Definition Language (DTDL) v2 — especificación de interfaces, telemetría, propiedades y comandos.
- ASHRAE TC 9.9 — Thermal Guidelines for Data Processing Environments (clases A1–A4, rango recomendado 18–27 °C).
- ANSI/TIA-942 — Telecommunications Infrastructure Standard for Data Centers (uso académico).
- UL 268 — Smoke Detectors for Fire Alarm Systems (rango de sensibilidad 0,5–4,0 %/ft).
- Hojas de datos: Sensirion SHT45, SHT31 y SDP810; Analog Devices/Maxim DS18B20; Aosong AM2302 (DHT22); Plantower PMS5003; Eastron SDM630-Modbus V2; Davis Instruments Vantage Pro2 (6152); NXP MFRC522.
- APIs: OpenWeather Current Weather Data 2.5; Open-Meteo Forecast API; Open-Meteo Air Quality API (Copernicus CAMS).
- Wokwi — ESP32 Wi-Fi Networking: <https://docs.wokwi.com/guides/esp32-wifi>